오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

JavaScript 기초

DAY 18

fetch API, JSON, 오류 처리와 종합 실습

서버에 데이터를 요청해 JSON으로 받아 화면에 그리고,
실패한 경우까지 try · catch · finally로 다룹니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1 **HTTP 요청과 응답을 읽을 수 있다**

메서드 · 주소 · 상태 코드를 알고, 개발자 도구 Network 패널에서 직접 확인합니다.

2 **fetch로 데이터를 받아 화면에 그릴 수 있다**

JSON을 JavaScript 값으로 바꾸고, 받은 배열을 createElement로 목록에 그립니다.

3 **실패까지 다루는 코드를 쓸 수 있다**

response.ok를 확인하고, try · catch · finally로 오류와 「불러오는 중」을 처리합니다.

오늘의 결과물

회원 목록을 불러오는 members.js와, JavaScript 기초를 모아 만드는 할 일 보기 todos.js

ROADMAP

오늘의 흐름

01

HTTP 요청과 응답

API · GET · POST
상태 코드 · Network

02

JSON

stringify · parse
response.json()

03

fetch

then · await · ok
POST · 실습 01

실습 포함

04

오류 처리와 종합

try · catch · finally
종합 복습 · 실습 02

실습 포함

앞의 두 섹션은 서버와 주고받는 약속(HTTP)과 데이터의 모양(JSON)입니다.
뒤의 두 섹션에서 fetch로 받아 화면에 그리고, 실패한 경우까지 처리합니다.

되짚기 · 01

브라우저의 요청, 서버의 응답

배운 곳	그때 배운 것	오늘
DAY 01 5장	주소를 넣으면 브라우저가 요청, 서버가 HTML을 응답	HTML 대신 데이터(JSON)를 받음
DAY 02 12장	method — get은 주소에, post는 본문에 값을 실음	fetch도 GET · POST로 보냄
DAY 02 15장	개발자 도구의 Network 탭에서 이 과정을 볼 수 있음	Network 패널로 직접 봄

오늘은 페이지 전체가 아니라 필요한 데이터만 요청합니다

DAY 17 11장의 '필요한 부분만 받아 바꾸는 비동기 통신'을 fetch로 직접 만듭니다.

await로 기다리는 Promise, catch로 받는 실패

```
// DAY 17 실습 02 — order.js의 serve
async function serve(menu) {
  addLog(`${menu} 주문 접수 중…`);
  try {
    await takeOrder(menu);
    addLog(`${menu} 확인 — 만드는 중…`);
    const drink = await makeDrink(menu);
    addLog(`${drink}! 픽업대로 오세요`);
  } catch (error) {
    addLog(`주문 실패 : ${error}`);
  }
}
```

fetch도 Promise

응답이 오면 이행되는 Promise —
그래서 await로 기다립니다.

거부되면 catch로

await한 Promise가 거부되면
그 줄에서 오류가 던져져
catch로 갑니다.

준 비

오늘의 셋팅

```
Day18/                          ← 같은 저장소 안에 만듭니다
  index.html                    ← 미리 받은 파일 (장표 예제)
  index.js                      ← 장표를 보며 함께 칩니다
  blank.html                    ← 39장 예제가 fetch로 받는 파일
  style.css                     ← 모두 함께 씁니다
  members.html · members.js     ← 실습 01
  todos.html · todos.js         ← 실습 02
$ git status
/* 오늘도 main에 바로 커밋합니다 */
```

인터넷이 필요합니다

연습용 서버에 실제로
요청을 보냅니다.
끊기면 실패 화면이 나옵니다.

Network 패널

개발자 도구의 Network에서
오간 요청과 응답을
Console과 함께 봅니다.

01

HTTP 요청과 응답

API · 요청과 응답 · Network 패널 · GET · POST · 상태 코드 · 연습용 API

API — 프로그램끼리 주고받는 약속



1기 자료의 정의

API (Application Programming Interface)

서로 다른 소프트웨어가 데이터를 주고받거나 기능을 부를 수 있게 정해 둔 인터페이스

정해진 방식으로 요청하면, 정해진 모양으로 응답이 옵니다

오늘 쓰는 API는 주소(URL)로 요청하고 JSON으로 응답하는 웹 API입니다.

요청은 메서드와 주소, 응답은 상태 코드

브라우저

fetch를 부른
내 JavaScript

① 요청 — 브라우저 → 서버

`GET`	메서드 — 어떻게 (가져오기)
`/users`	주소 — 무엇을
헤더	덧붙이는 정보 · POST면 본문도

→

서버

jsonplaceholder
.typicode.com

←

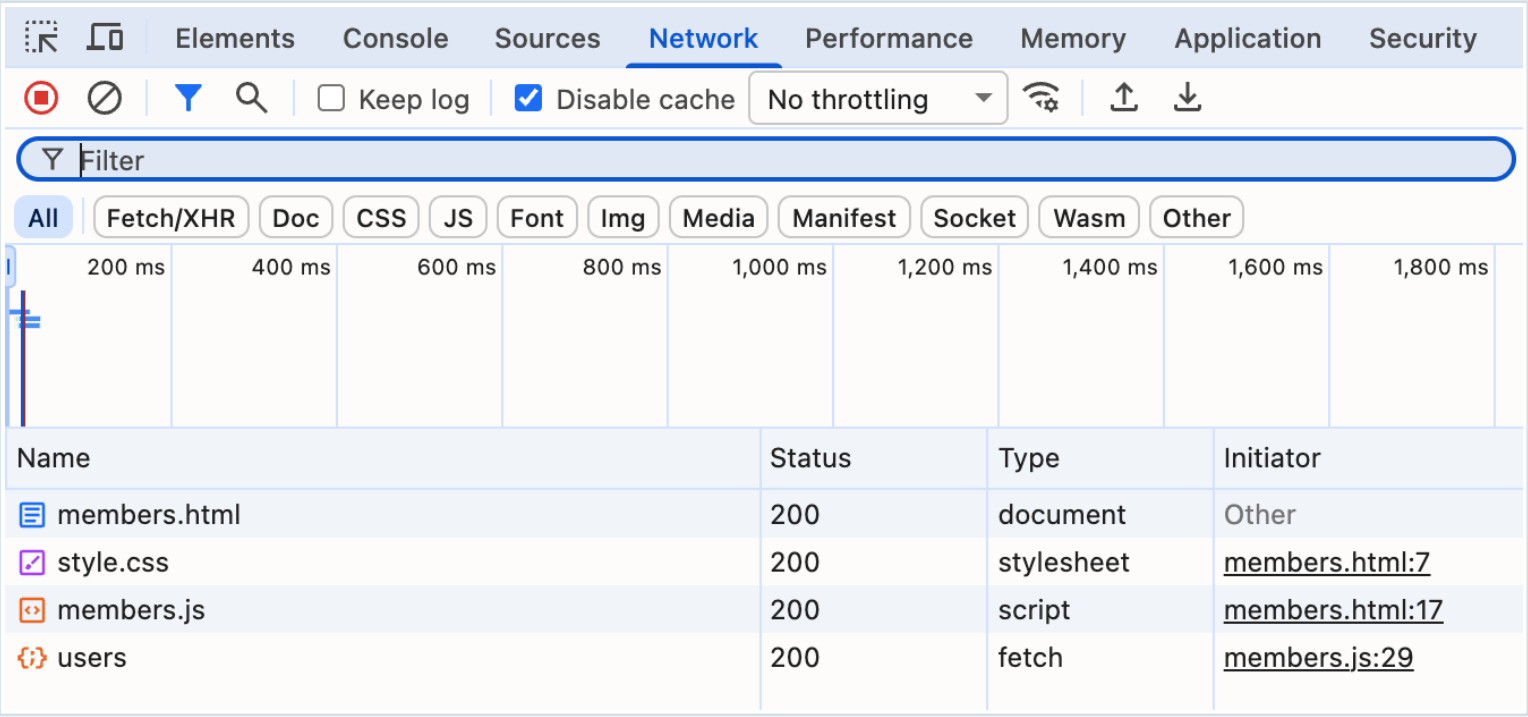
② 응답 — 서버 → 브라우저

`200 OK`	상태 코드 — 결과
헤더	덧붙이는 정보 (본문의 형식 등)
본문	받은 데이터 — 오늘은 JSON 글자

요청은 '어떻게 · 무엇을', 응답은 '결과 · 내용'

메서드와 주소를 정해 보내면, 상태 코드와 본문이 돌아옵니다.

요청 목록 — 한 줄이 요청 하나

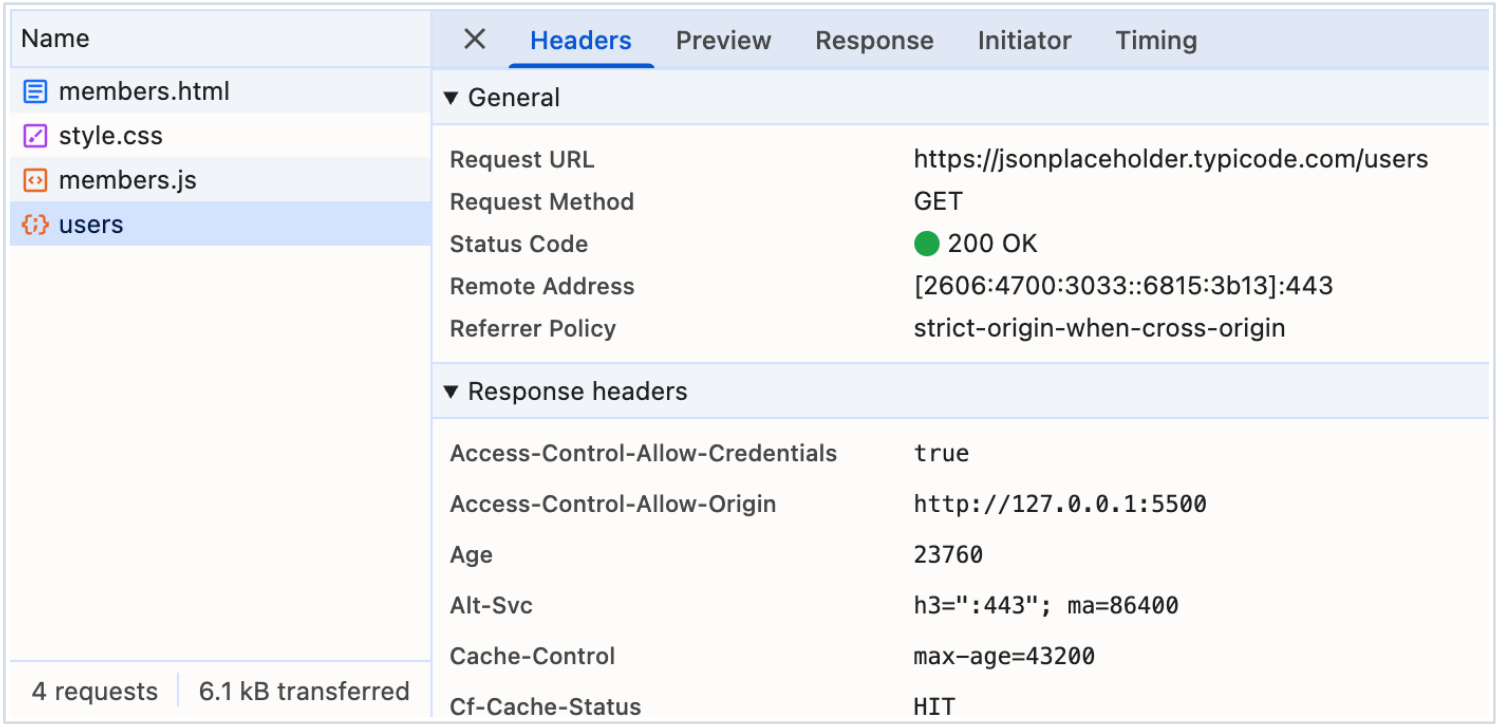


실습 01에서 「불러오기」를 누른 뒤 — 맨 아래 줄이 fetch 요청

열 읽는 법

Name — 요청한 것
Status — 상태 코드
Type — fetch는 코드가 보냄
Initiator — 요청을 보낸 줄

줄을 누르면 보이는 요청과 응답



users 줄을 누른 화면 — Headers 탭

읽는 법

Request URL — 요청한 주소
Request Method — GET
Status Code — 200 OK
Response headers
— 응답에 붙은 정보

가져올 때 GET, 보낼 때 POST

GET

데이터를 가져옴

주소창에 주소를 넣거나
링크를 눌러도 GET
fetch는 정하지 않으면 GET

POST

데이터를 서버에 보냄

DAY 02의 form method="post"
회원가입 · 글쓰기처럼
보낼 값은 본문(body)에

fetch(주소)만 쓰면 GET — POST는 method를 정해 보냅니다

POST는 섹션 03에서 method · headers · body를 정해 직접 보내 봅니다.

상태 코드 — 세 자리 숫자로 결과를 알림

범위	뜻	자주 보는 코드
200~299	성공	200 OK · 201 Created (새로 만들어짐)
300~399	다른 주소로 (리다이렉트)	301 Moved Permanently (주소가 옮겨 감)
400~499	요청 쪽 문제	404 Not Found (찾을 수 없음)
500~599	서버 쪽 문제	500 Internal Server Error (서버 오류)

200번대면 성공 — 섹션 03의 response.ok가 이것을 알려 줍니다

1기 자료 27장의 「200(성공), 404(찾을 수 없음), 500(서버 에러)」를 범위로 넓혀 봤습니다.

JSONPlaceholder — 연습용 가짜 API

```
// 기본 주소
https://jsonplaceholder.typicode.com

// 뒤에 붙이는 길 — 돌려주는 것
/users              회원 10명
/users/1            1번 회원 한 명
/posts              글 100개
/todos              할 일 200개
/todos?userId=1     1번 회원의 할 일만
```

?userId=1 — 원하는 것만 고르는 쿼리

DAY 02 12장의 get처럼 주소 뒤에 ?이름=값을 붙입니다.
실습 02는 이 쿼리로 회원별 할 일을 받습니다.

pretty print 적용 ☐


```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
}
```


주소창에 …/users/1을 넣어 연 화면

여기까지 정리

1	API	프로그램끼리 데이터를 주고받는 약속
2	요청	메서드(GET · POST)와 주소 — 보낼 값이 있으면 본문
3	응답	상태 코드와 본문 — 본문은 오늘 JSON 글자
4	GET · POST	서버에서 데이터를 가져올 때 GET, 서버에 데이터를 보낼 때 POST
5	상태 코드	200 요청 성공 · 404 찾을 수 없음 · 500번대 서버 오류

다음은 응답 본문의 모양 — JSON을 봅니다

02

JSON

JSON의 모양 · Response 탭 · JSON.stringify · JSON.parse · 잘못된 JSON · response.json()

JSON — 객체 모양을 본뜬 글자 데이터

```
// JavaScript 객체 — 코드 안의 값
const user = {
  name: "김오즈",
  age: 20,
};

// JSON — 같은 내용을 글자로
{
  "name": "김오즈",
  "age": 20
}
```

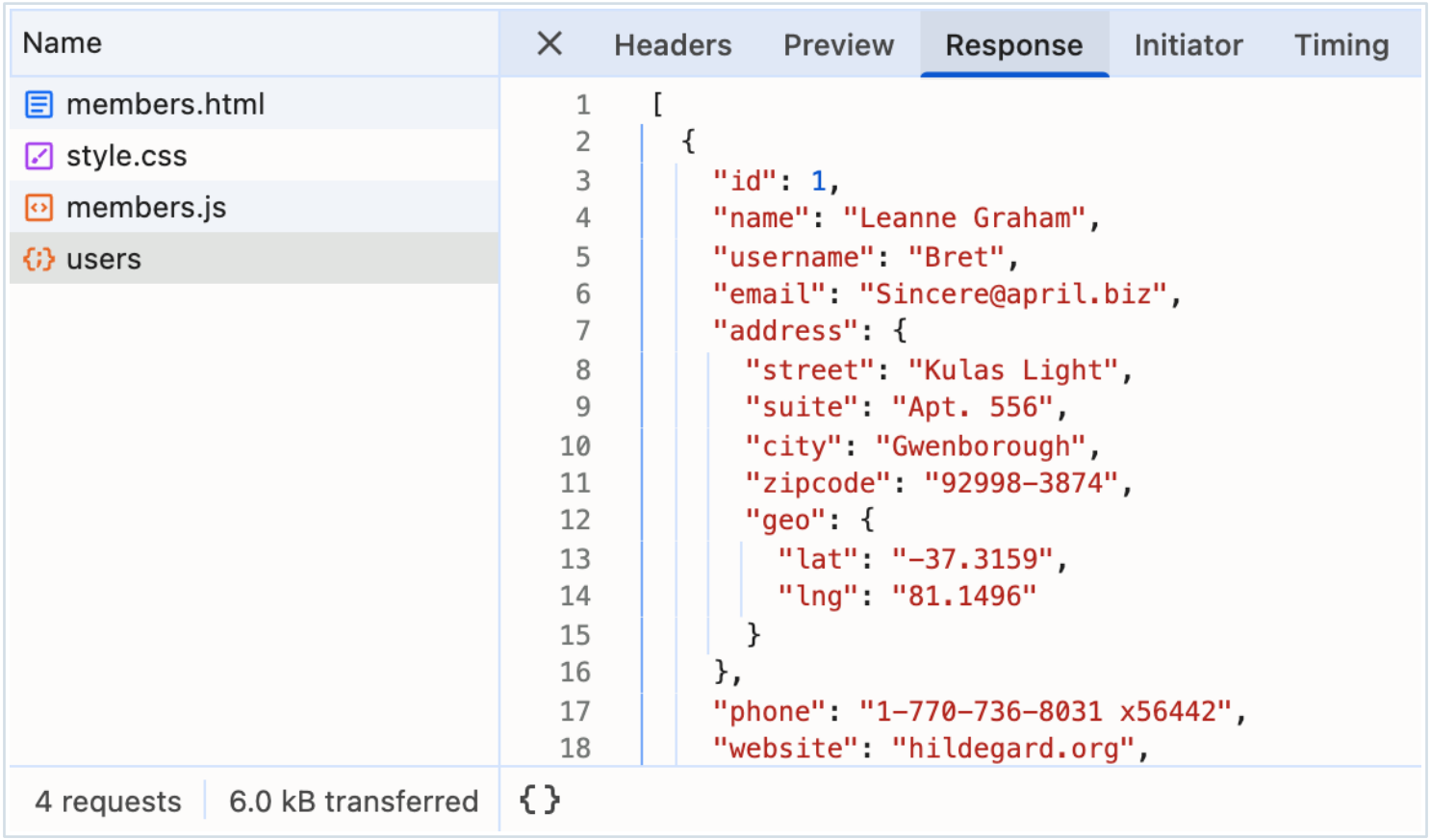
JSON의 규칙

키(이름)는 반드시 큰따옴표
글자 값도 큰따옴표 — 작은따옴표는 안 됨
마지막 쉼표 · 주석은 안 됨
값 — 글자 · 숫자 · true · false · null
· 배열 · 객체
함수 · undefined는 담을 수 없음

JSON(JavaScript Object Notation) — 서버와 주고받는 글자 데이터

모양은 객체와 비슷하지만 글자입니다 — 받은 뒤 JavaScript 값으로 바꿔 씁니다.

서버가 보낸 JSON — Response 탭




```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "username": "Bret",
    "email": "Sincere@april.biz",
    "address": {
      "street": "Kulas Light",
      "suite": "Apt. 556",
      "city": "Gwenborough",
      "zipcode": "92998-3874",
      "geo": {
        "lat": "-37.3159",
        "lng": "81.1496"
      }
    },
    "phone": "1-770-736-8031 x56442",
    "website": "hildegard.org",
```

읽는 법

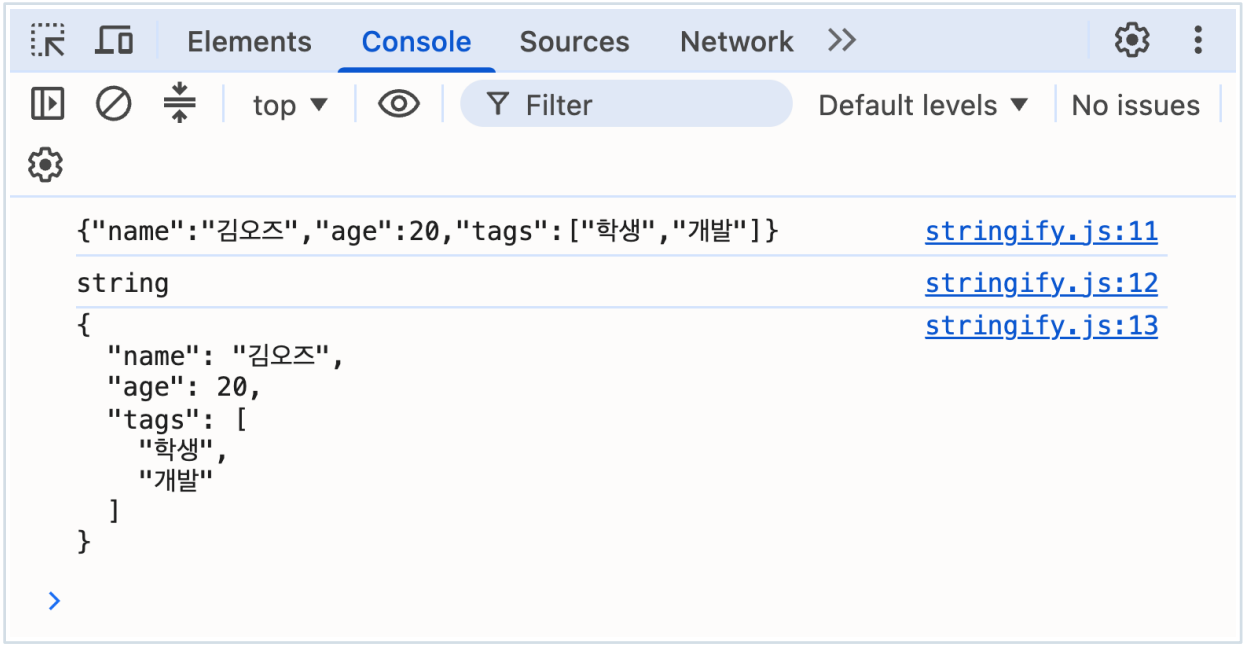
[ ] — 회원 10명이 든 배열
{ } — 회원 한 명 (객체)
"키": 값 — 키는 큰따옴표
address — 객체 안의 객체

users 줄 → Response 탭 — 받은 글자 그대로

JSON.stringify — 값을 JSON 글자로 바꾸기

```
const user = {
  name: "김오즈",
  age: 20,
  tags: ["학생", "개발"],
  greet() {
    return "안녕하세요";
  },
};

const text = JSON.stringify(user);
console.log(text);
console.log(typeof text);
console.log(JSON.stringify(user, null, 2));
```



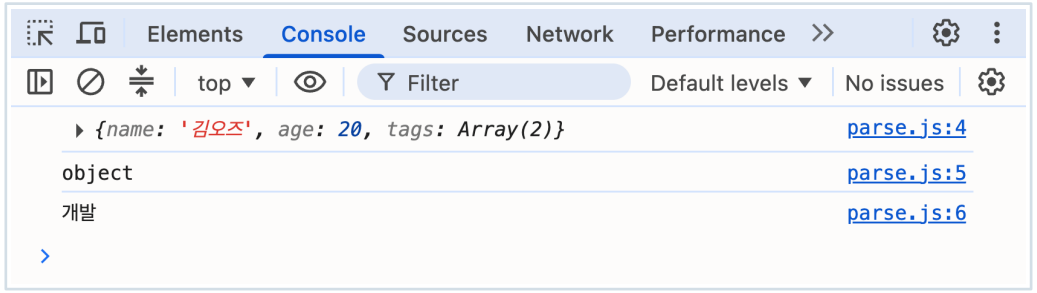
JSON.stringify(값)

객체 → JSON 문자열 · 함수(greet)는 빠짐
셋째 칸 2 — 두 칸씩 들여 씀

JSON.parse — JSON 글자를 값으로 바꾸기

```
const text = '{"name":"김오즈","age":20,"tags":["학생","개발"]}';
const user = JSON.parse(text);

console.log(user);
console.log(typeof user);
console.log(user.tags[1]);
```



JSON.parse(글자)

JSON 문자열 → 객체

typeof는 object

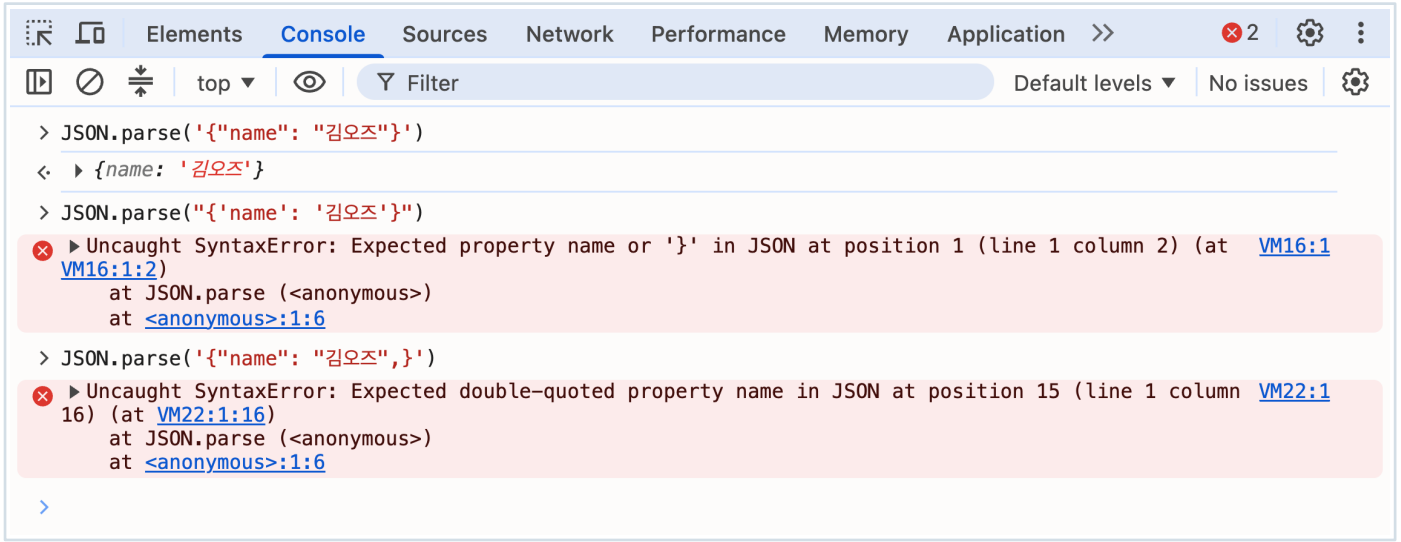
꺼내 쓰기 — user.tags[1]

규칙을 어기면 JSON.parse에서 SyntaxError

```
// Console에 한 줄씩 칩니다
JSON.parse('{"name": "김오즈"}')
// 큰따옴표 → 객체

JSON.parse("{'name': '김오즈'}")
// 작은따옴표 → SyntaxError

JSON.parse('{"name": "김오즈",}')
// 마지막 쉼표 → SyntaxError
```



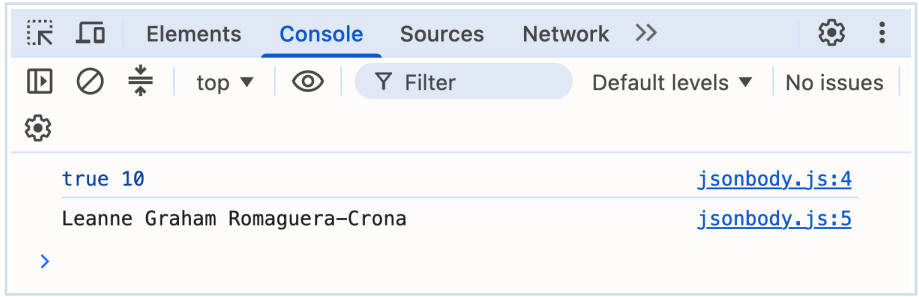
읽는 법

position — 막힌 글자 자리 (0부터)
작은따옴표 — position 1
마지막 쉼표 — position 15

받은 본문은 response.json()으로 읽기

```
async function load() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  const users = await response.json();
  console.log(Array.isArray(users), users.length);
  console.log(users[0].name, users[0].company.name);
}

load();
```



response.json()

본문을 읽어 JavaScript 값으로
Promise를 돌려줌 — await 한 번 더
회원 10명 → 길이 10인 배열

여기까지 정리

1	JSON	객체 모양을 본뜬 글자 데이터 — 키와 글자는 큰따옴표
2	JSON.stringify	JavaScript 값을 JSON 문자열로 바꿈
3	JSON.parse	JSON 문자열을 JavaScript 값으로 바꿈
4	잘못된 JSON	작은따옴표 · 마지막 쉼표 — JSON.parse에서 SyntaxError
5	response.json()	응답 본문을 읽어 JavaScript 값으로 — Promise를 돌려줌

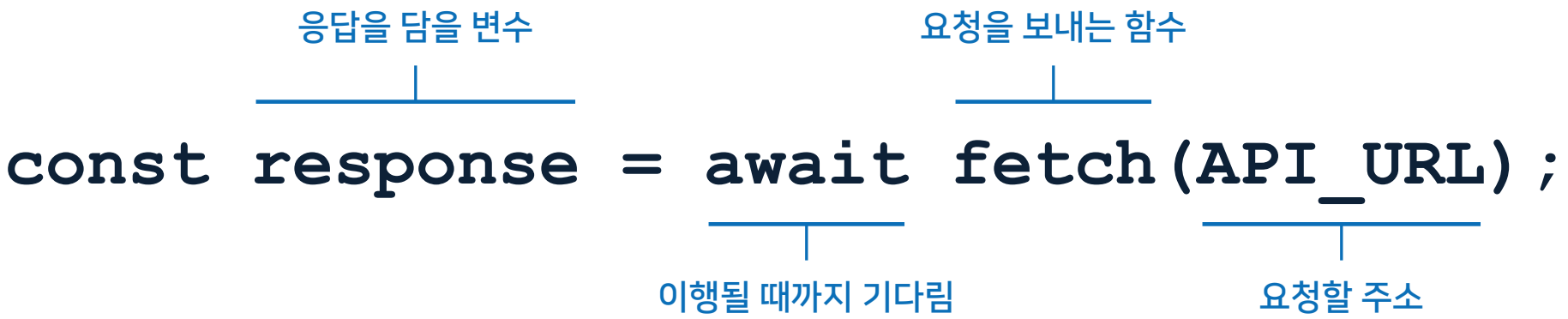
다음은 fetch — 요청을 보내고 응답을 받는 코드

03

fetch

fetch 나눠 읽기 · then · await · status와 ok · 404는 거부가 아님 · POST · 실습 01

한 줄 나눠 읽기



fetch(주소) — 요청을 보내고, 응답이 오면 이행되는 Promise를 돌려줌

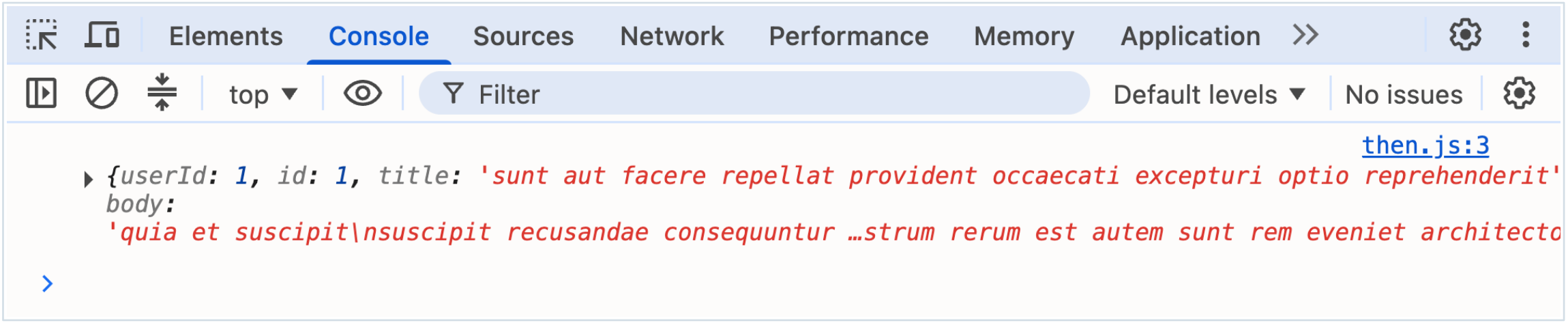
이행된 값은 응답(Response) 객체 — 본문은 response.json()으로 한 번 더 읽습니다.

1기 자료의 첫 예제 — then으로 받기

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => console.error("에러:", error));
```

읽는 법

첫 then — 본문을 JSON으로 읽음
둘째 then — 읽은 객체를 받음
요청이 실패하면 catch



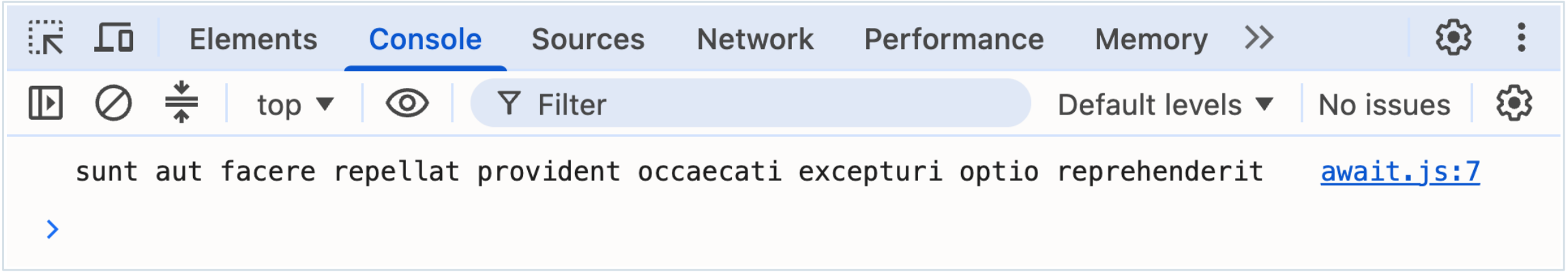
await로 받기 — 위에서 아래로 읽히는 코드

```
async function fetchPost() {
  const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
  const data = await response.json();
  return data.title;
}

fetchPost().then((title) => console.log(title));
```

then 대신 await

위에서 아래로 차례대로 읽힘
await 두 번 — fetch와 json()
async 함수의 값은 then으로

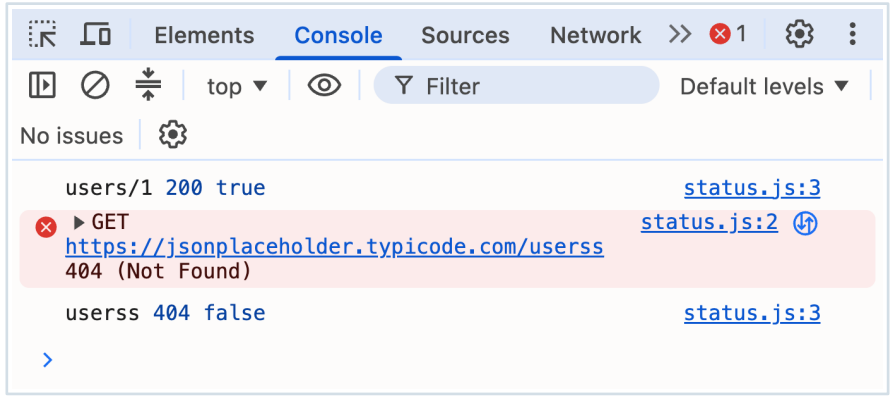


response.status와 response.ok

```
async function check(path) {
  const response = await fetch(`https://jsonplaceholder.typicode.com/${path}`);
  console.log(path, response.status, response.ok);
}

async function main() {
  await check("users/1");
  await check("userss");
}

main();
```

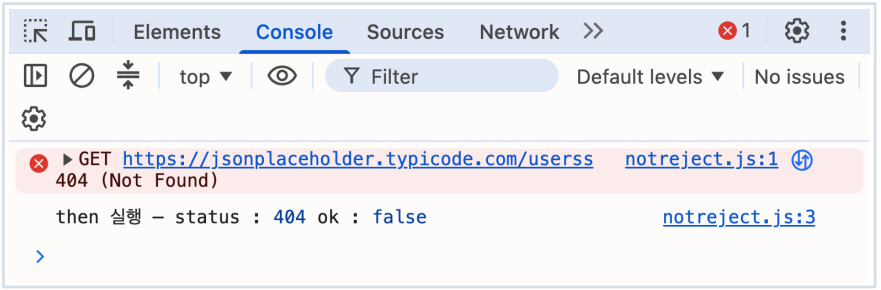


응답에 들어 있는 것

status — 상태 코드 숫자
ok — 200~299이면 true
없는 주소에도 응답은 옴 — 404

없는 주소여도 이행되는 fetch

```
fetch("https://jsonplaceholder.typicode.com/userss")
  .then((response) => {
    console.log("then 실행 — status :", response.status, "ok :", response.ok);
  })
  .catch((error) => {
    console.log("catch 실행 —", error.message);
  });
```



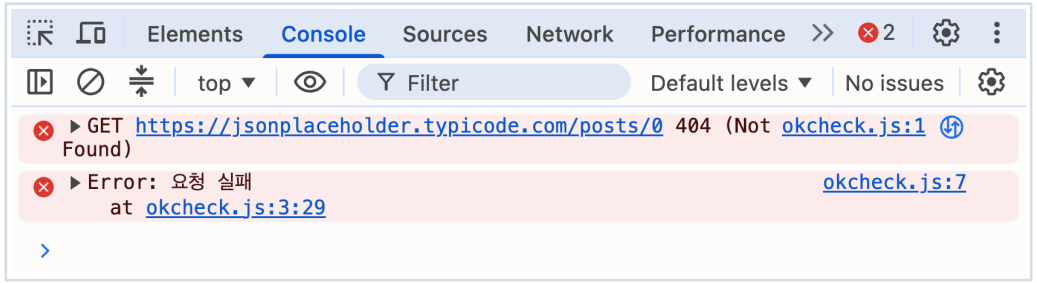
Console — then은 실행, catch는 실행 안 됨

Name	Status	Type	Initiator
notreject.html	200	document	Other
style.css	200	stylesheet	notreject.html:7
notreject.js	200	script	notreject.html:14
userss	404	fetch	notreject.js:1

Network — userss는 404여도 응답은 왔음 → fetch는 이행 · ok를 직접 확인 (다음 장)

ok가 아니면 throw — catch로 보내기

```
fetch("https://jsonplaceholder.typicode.com/posts/0")
  .then((response) => {
    if (!response.ok) throw new Error("요청 실패");
    return response.json();
  })
  .then((data) => console.log(data.title))
  .catch((error) => console.error(error));
```



ok가 false면 throw

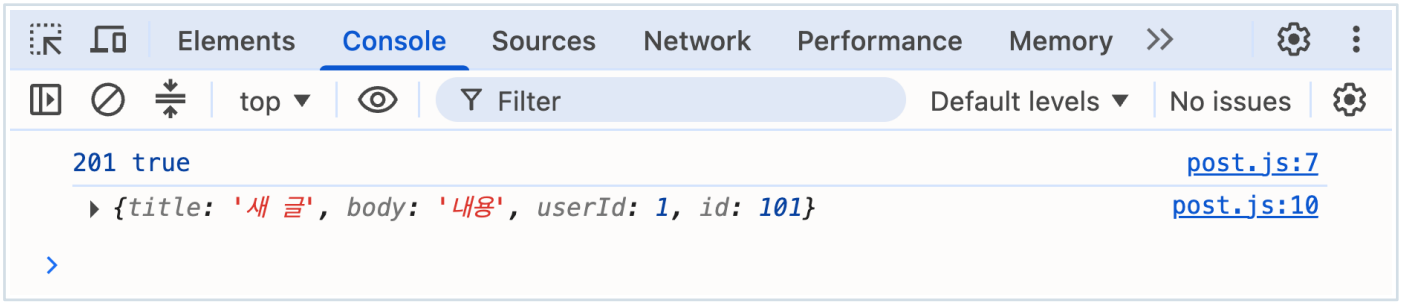
throw — 오류를 던져 catch로
new Error("글") — 오류 객체
사이의 then은 건너뜀

POST로 보내기 — 둘째 칸의 옵션 객체

```
fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "새 글", body: "내용", userId: 1 }),
})
  .then((response) => {
    console.log(response.status, response.ok);
    return response.json();
  })
  .then((data) => console.log(data))
  .catch((error) => console.error(error));
```

옵션 객체

method — 보내는 방식
headers — 본문이 JSON임을 알림
body — JSON.stringify로 글자로



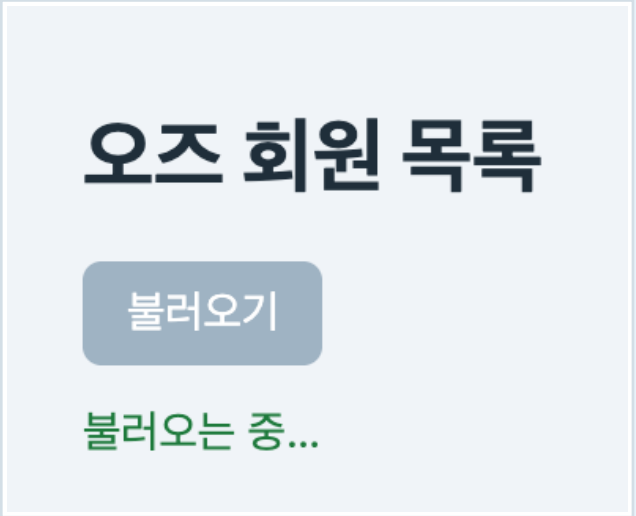
실습 01

회원 목록 불러오기

members.html은 그대로 두고 members.js만 써서, 누르면 회원 10명을 받아 목록으로 그립니다.
fetch · response.ok · response.json() · try…catch…finally와 DAY 15의 createElement를 씁니다.

만들 것

① createMember — 한 명으로 li 하나
② render — 목록을 비우고 다시 그림
③ loadMembers — 요청 · ok · json
④ catch — 실패하면 빨간 안내 문구
⑤ finally — 버튼 다시 켜기
⑥ 불러오기 버튼에 click 연결



누른 직후 —「불러오는 중…」

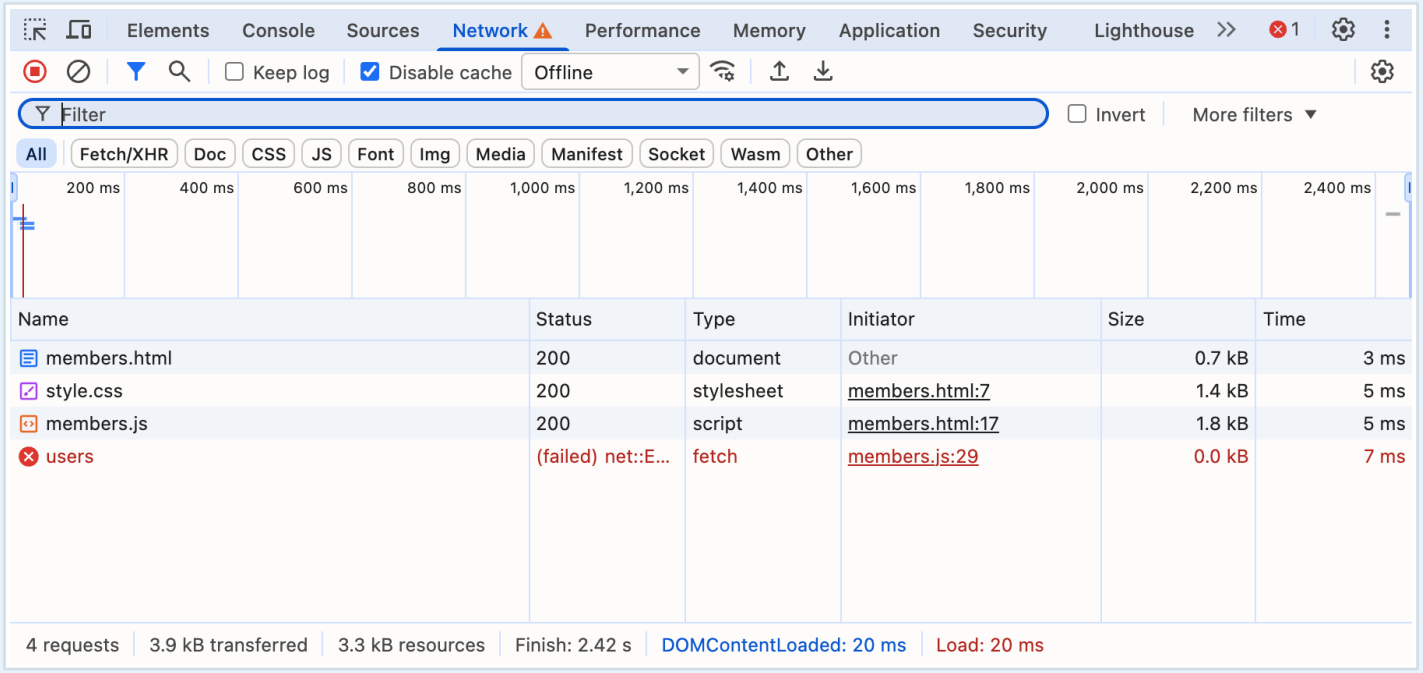


받은 뒤 — 10명과「10명을 불러왔습니다」

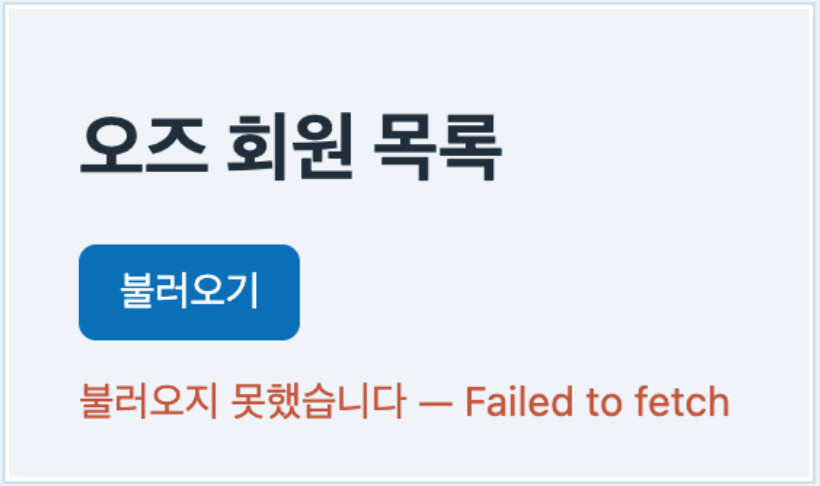
실습 01

실패 화면도 직접 만들기

Network 패널의 No throttling 목록에서 Offline을 고르고 「불러오기」를 누릅니다.
끝나면 No throttling으로 되돌립니다 — 그대로 두면 다시 열어도 요청이 실패합니다.



Network — Offline에서 users 요청이 (failed)



화면 — 빨간 안내, 버튼은 다시 켜짐

실습 01 · 확인

틀린 곳이 먼저 드러나는 화면의 안내 문구

바꿀 곳	결과 — 화면의 안내 문구
① 5번 줄 주소의 users를 userss로	「불러오지 못했습니다 — 404 오류」— ok 검사가 한 일
② ①에 이어 30~32번 줄 앞에 // 붙이기	「… — users.forEach is not a function」
③ 29번 줄의 await 빼기	「… — undefined 오류」— Console에는 빨간 줄 없음
④ 33번 줄의 await 빼기	「… — users.forEach is not a function」
⑤ 41번 줄 앞에 // 붙이기	목록은 나오지만 버튼이 회색으로 남음

not a function이 보이면 — 그 값이 배열인지, await와 ok 검사부터 봅니다

여기까지 정리

1	**fetch(주소)**	요청을 보내고, 응답이 오면 이행되는 Promise를 돌려줌
2	**await 두 번**	fetch로 응답을, response.json()으로 본문을 받음
3	**status · ok**	상태 코드 숫자와, 200~299이면 true인 값
4	**404도 이행**	ok가 false면 throw로 catch에 보냄
5	**POST**	method · headers · body(JSON.stringify)를 정해 보냄

다음은 실패를 다루는 try · catch · finally와 기초 종합

04

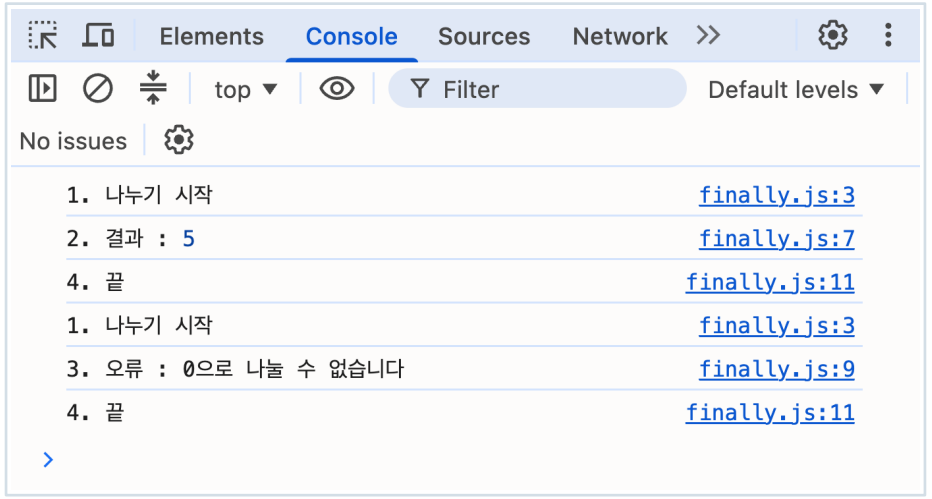
오류 처리와 종합

finally · 네트워크 오류 · 실패의 세 갈래 · Promise.all · 기초 종합 · 실습 02

성공해도 실패해도 마지막에 도는 finally

```
function divide(a, b) {
  try {
    console.log("1. 나누기 시작");
    if (b === 0) {
      throw new Error("0으로 나눌 수 없습니다");
    }
    console.log("2. 결과 :", a / b);
  } catch (error) {
    console.log("3. 오류 :", error.message);
  } finally {
    console.log("4. 끝");
  }
}

divide(10, 2);
divide(10, 0);
```



finally

성공해도 실패해도 마지막에 실행

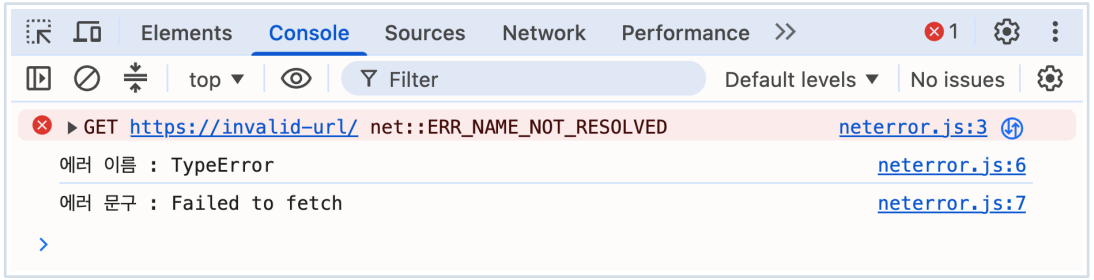
실습 01 — 버튼 다시 켜기

실습 02 — 화면 다시 그리기

응답이 아예 없으면 거부되는 fetch

```
async function load() {
  try {
    const response = await fetch("https://invalid-url");
    console.log(response.status);
  } catch (error) {
    console.log("에러 이름 :", error.name);
    console.log("에러 문구 :", error.message);
  }
}

load();
```



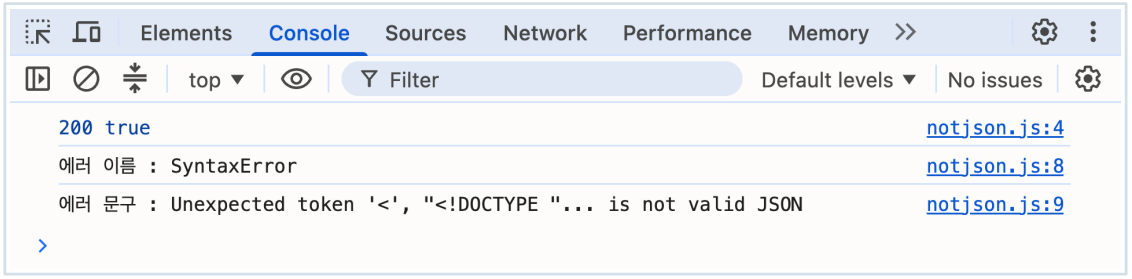
응답이 아예 없으면

fetch의 Promise가 거부 — TypeError
Chrome의 문구 — Failed to fetch
await 줄에서 던져져 catch로

본문이 JSON이 아니면 거부되는 json()

```
async function load() {
  try {
    const response = await fetch("blank.html");
    console.log(response.status, response.ok);
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("에러 이름 :", error.name);
    console.log("에러 문구 :", error.message);
  }
}

load();
```



200이어도 실패할 수 있음

본문이 HTML — JSON이 아님
json()의 Promise가 거부
SyntaxError가 catch로

실패는 세 곳에서, 받는 catch는 하나

요청이 닿지 못함

fetch가 거부

도메인 오타 · 인터넷 끊김
TypeError — Failed to fetch
await 줄에서 catch로

서버가 오류로 응답

ok가 false

404 · 500 — 응답은 옴
그대로 두면 이행됨
throw로 catch에 보냄

본문이 JSON이 아님

json()이 거부

HTML 같은 다른 글자
SyntaxError
await 줄에서 catch로

try 안에서 await하고, ok를 확인하면 세 가지 실패가 모두 catch로 모입니다

catch에서는 사용자에게 알리고, finally에서는 성공 · 실패와 상관없이 할 일을 합니다.

Promise.all — 두 요청을 함께 기다리기

```
// index.js 1~7번 줄 (8번 줄부터는 다음 장)
async function displayUserPosts() {
  const outputEl = document.querySelector("#output");
  try {
    const [user, posts] = await Promise.all([
      fetch("https://jsonplaceholder.typicode.com/users/2").then((res) => res.json()),
      fetch("https://jsonplaceholder.typicode.com/posts?userId=2").then((res) => res.json()),
    ]);
```

구조 분해 — 처음

const [user, posts]
배열의 첫째 · 둘째
값을 각각 변수에

Name	Status	Type	Initiator	Size	Time
userposts.html	200	document	Other	0.5 kB	5 ms
style.css	200	stylesheet	userposts.html:7	1.4 kB	3 ms
userposts.js	200	script	userposts.html:15	1.0 kB	3 ms
2	200	fetch	userposts.js:5	1.0 kB	219 ms
posts?userId=2	200	fetch	userposts.js:6	1.7 kB	219 ms

5 requests | 5.6 kB transferred | 5.6 kB resources | Finish: 238 ms | DOMContentLoaded: 27 ms | Load: 28 ms

Network — 두 fetch 요청 (users/2 · posts?userId=2)

읽는 법

두 요청 모두 219ms
모두 끝난 때 238ms
차례로가 아니라 함께

받은 회원과 글을 화면에 그리기

```
// index.js 8~24번 줄
    const nameEl = document.createElement("h2");
    nameEl.textContent = user.name;
    outputEl.appendChild(nameEl);

    const listEl = document.createElement("ul");
    posts.forEach((post) => {
      const itemEl = document.createElement("li");
      itemEl.textContent = post.title;
      listEl.appendChild(itemEl);
    });
    outputEl.appendChild(listEl);
  } catch (error) {
    console.error("에러:", error);
  }
}

displayUserPosts();
```

회원과 글

Ervin Howell

- et ea vero quia laudantium autem
- in quibusdam tempore odit est dolorem
- dolorum ut in voluptas mollitia et saepe quo animi
- voluptatem eligendi optio
- eveniet quod temporibus
- sint suscipit perspiciatis velit dolorum rerum ipsa laboriosam odio
- fugit voluptas sed molestias voluptatem provident
- voluptate et itaque vero tempora molestiae
- adipisci placeat illum aut reiciendis qui
- doloribus ad provident suscipit at

2번 회원의 이름과 글 10개

1기의 innerHTML 대신 createElement · textContent

한 파일에 모은 JavaScript 기초 아홉 날

DAY	배운 것	실습 02 todos.js에서
DAY 10 · 11	변수 · 비교 · 조건문	let todos · let view, if (view === "done")
DAY 12	함수 · 화살표 함수	getVisible · render, (todo) => …
DAY 13	배열 · 객체	todos 배열, todo.title · todo.completed
DAY 14	filter · forEach	끝낸 일 · 남은 일 고르기, li 만들기
DAY 15	createElement · classList	li를 만들고, 끝낸 일에 done 클래스
DAY 16	click · change	보기 버튼, 회원 고르기
DAY 17 · 18	async · await · fetch · finally	할 일 받아 오기, 실패해도 다시 그리기

새 문법은 하나도 없습니다 — 배운 것을 한 파일에 모읍니다

데이터를 바꾸고, 화면은 render로 다시

1 불러오기

loadTodos — fetch로
받은 배열을 todos에

2 고르기

getVisible — view에
따라 filter로 고름

3 그리기

render — 목록을 비우고
고른 것만 li로

4 바꾸기

버튼 · select가
view나 todos를 바꾸고
render를 다시 부름

화면을 직접 고치지 않고, 데이터(todos · view)를 바꾼 뒤 다시 그립니다

그래서 보기 버튼은 서버에 다시 묻지 않고, 회원을 바꿀 때만 새로 요청합니다.

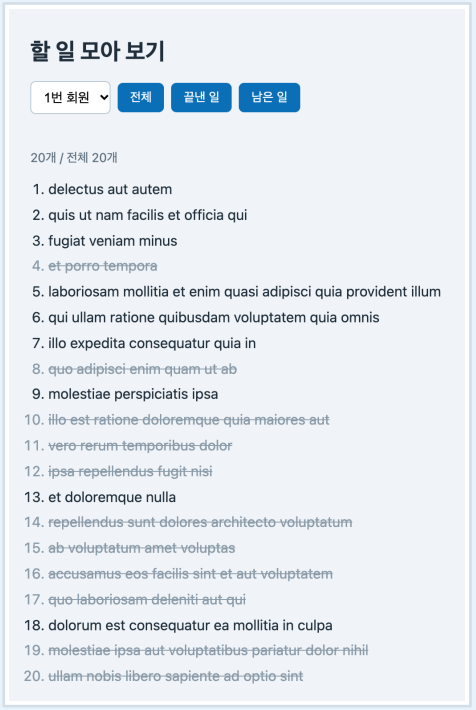
실습 02

할 일 모아 보기

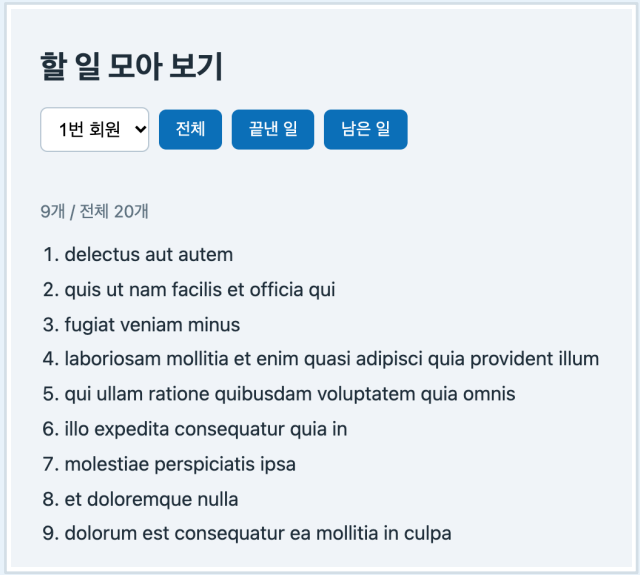
todos.html은 그대로 두고 todos.js만 써서, 회원의 할 일을 받아 전체 · 끝낸 일 · 남은 일로 나눠 봅니다.
DAY 10부터 오늘까지 배운 배열 · filter · createElement · 이벤트 · fetch를 모두 씁니다.

만들 것

① getVisible — 보기에 맞게 filter
② render — li · done 클래스 · 개수
③ loadTodos — 받기, 실패면 빈 배열
④ 회원을 바꾸면 다시 불러오기
⑤ 보기 버튼 — view를 바꾸고 render
⑥ 처음 열 때 1번 회원



처음 열면 — 1번 회원의 20개



남은 일 — 9개 / 전체 20개

실습 02

일부러 틀려 보기

완성한 todos.js를 한 곳씩 바꿔 저장하고, 결과가 어떻게 바뀌는지 본 뒤 되돌립니다.
②만 Console에 코드 오류가 뜹니다. 나머지는 화면을 보고 찾습니다.

바꿀 곳	결과
① 55번 줄 render(); 앞에 // 붙이기	처음에 빈 화면 — 보기 버튼을 눌러야 그려지고, 회원을 바꿔도 그대로
② 48번 줄의 await 빼기	빈 화면 — Console에 Uncaught (in promise) TypeError
③ 19번 줄의 ! 빼기	남은 일을 눌러도 줄 그은 할 일만 — 빨간 줄 없음
④ 44번 줄 주소의 todos를 todo로	「불러오지 못했습니다 — 404 오류」와 「0개 / 전체 0개」

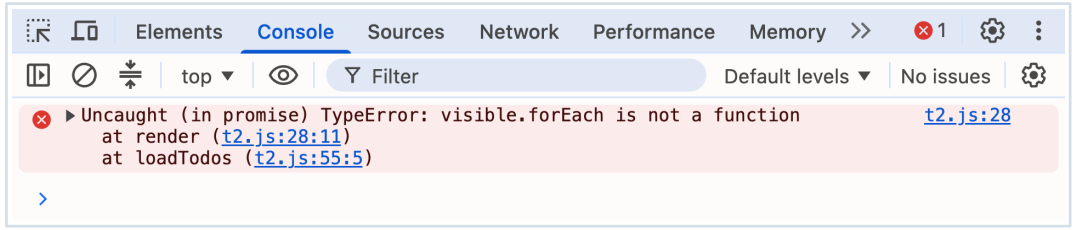
네 가지를 다 보고 되돌렸으면 커밋합니다 — 실습 02 끝

await를 빠뜨리면 — 빈 화면과 빨간 줄

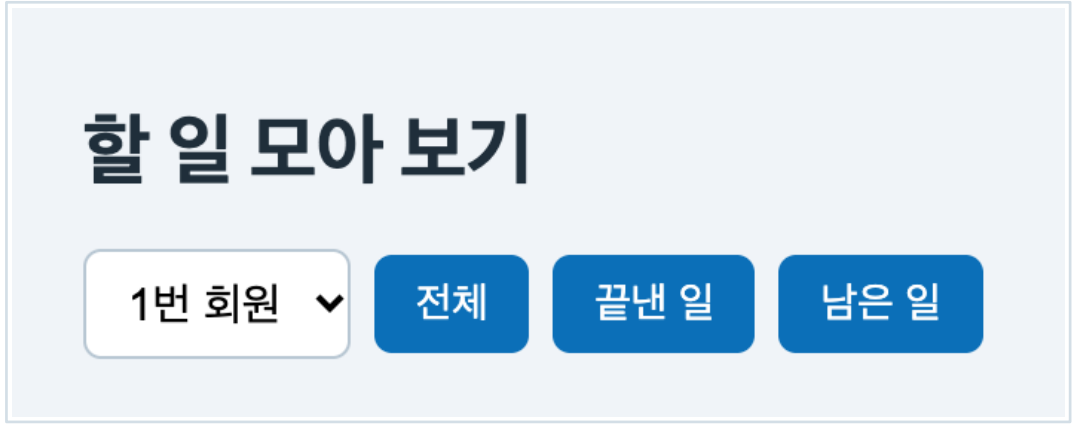
```
// todos.js 45~56번 줄 (일부러 틀린 곳 ②)
    if (!response.ok) {
      throw new Error(`${response.status} 오류`);
    }
    todos = response.json();   // await를 빠뜨림
    statusEl.textContent = "";
  } catch (error) {
    todos = [];
    statusEl.textContent = `불러오지 못했습니다 — ${error.message}`;
    statusEl.classList.add("error");
  } finally {
    render();   // 여기서 TypeError
  }
```

확인하는 법

is not a function이면 그 값이 배열인지 봅니다.
Promise가 들어 있으면 await가 빠진 것입니다.



Console — finally 안의 오류는 받는 곳이 없음



화면 — 목록도 개수도 없음

점검

데이터를 받을 때 자주 나오는 세 가지

!

404인데 then으로 가요

fetch는 응답이 오면 이행됩니다.
ok가 false면
throw로 catch에 보냅니다.

!

… is not a function

배열이 아닌 값에 forEach.
json() 앞의 await나
ok 검사가 빠졌는지 봅니다.

!

Failed to fetch

요청이 서버에 닿지 못했습니다.
주소의 도메인 · 인터넷 연결,
Offline 설정을 봅니다.

실패는 화면 · Console · Network 세 곳에서 확인합니다

Network 패널의 Status와 Console의 줄 번호를 함께 보면 어디서 틀렸는지 좁혀집니다.

WRAP UP

오늘 정리

개념

요청은 메서드 · 주소, 응답은 상태 코드 · 본문

JSON — 큰따옴표를 쓰는 글자 데이터

fetch는 Response로 이행되는 Promise

404 · 500은 거부가 아님 — ok로 확인

finally는 성공해도 실패해도 실행

Promise.all로 여러 요청을 함께 기다림

직접 쓸 수 있어야 하는 것

fetch와 await로 데이터 받기

response.json()으로 본문 읽기

JSON.stringify · JSON.parse로 바꾸기

ok를 확인하고 throw로 catch에 보내기

try · catch · finally로 화면 상태 처리

받은 배열을 createElement로 그리기

참고교재

교재에서 함께 읽을 곳

오늘 수업과 이어지는 장입니다. 복습할 때 장 번호로 찾아 읽습니다.

길벗캠퍼스

『백엔드 입문자를 위한 모던 자바스크립트 & Node.js』

이창현 지음

9장. ES2015+ 문법

위키북스

『모던 자바스크립트 Deep Dive』

이웅모 지음

42장. 비동기 프로그래밍
45장. 프로미스
46장. 제너레이터와 async/await

NEXT

DAY 19

고급 함수와 고차 함수

스코프 · 클로저 · this로 함수가 동작하는 원리를 보고,
map · filter · reduce로 데이터를 다룹니다.

수고하셨습니다.